Lesson 14 Teacher's Key: Raising the Stakes

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

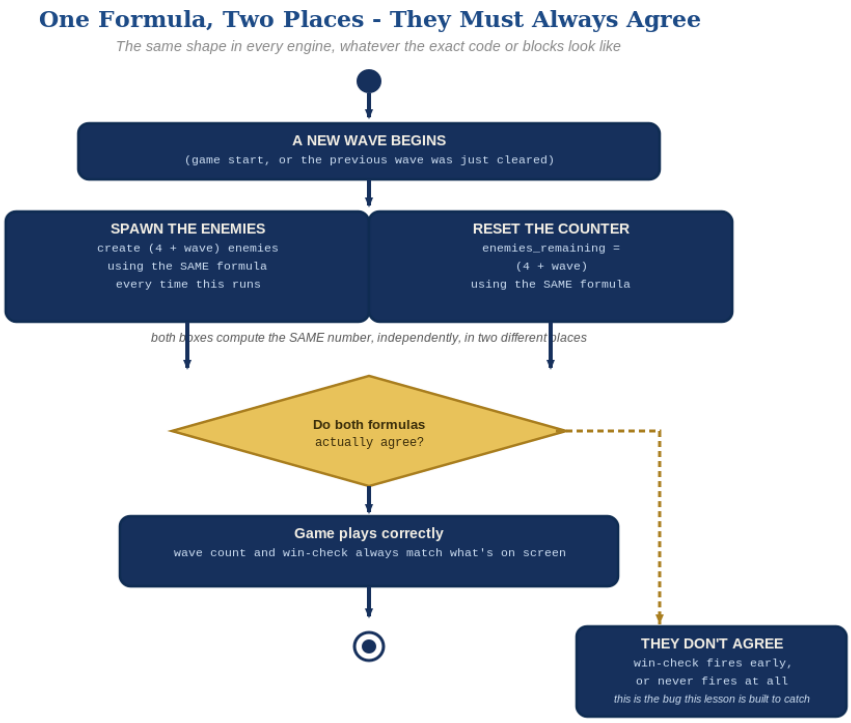
Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as C# expressions inside EnemySpawner.cs's SpawnEnemies() method.

Level 1 - Full Worked Solution

- int count = 4 + GameManager.wave computes the count once, then reuses it for both the loop and enemiesRemaining - a single source of truth by construction, not by discipline alone.
- No new fields are needed for Level 1 - this is purely a formula change inside a method that already exists.
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.

Highlighted lines are new this week; everything else is code you already have.

```
// EnemySpawner.cs:
public void SpawnEnemies() {
    int count = 4 + GameManager.wave;
    for (int enemyIndex = 0; enemyIndex < count; enemyIndex++) {
        GameObject newEnemy = Instantiate(enemyPrefab);
        // ...existing per-enemy positioning...
    }
    GameManager.enemiesRemaining = count;
}
```



Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

Why This Structure

- Storing count in a local variable (rather than repeating the (4 + GameManager.wave) expression twice) is good practice worth naming explicitly - it guarantees the loop and the reset value can never disagree, since they read the same computed value.
- For Level 2, setting each enemy's speed field right after Instantiate() (rather than changing the prefab's own default) keeps the scaling logic entirely inside SpawnEnemies() - Enemy.cs itself stays exactly as simple as Lesson 7 left it.

Level 2 / Level 3 Talking Points

- Ask students why storing count in a local variable is safer than writing the formula twice - a good, concrete illustration of avoiding duplicated logic.
- For students comparing engines: point out that GameMaker's identical (4 + wave) formula, expressed in GML instead of C#, is conceptually the exact same idea - a good cross-engine parallel.

Level 2 - complete code:

Highlighted lines are the Level 2 additions on top of the Level 1 code above.

```
// EnemySpawner.cs - SpawnEnemies() (Level 2 addition, on top of the Level
1 code above - speed set on each new instance):
public void SpawnEnemies() {
    int count = 4 + GameManager.wave;
```

```
    for (int enemyIndex = 0; enemyIndex < count; enemyIndex++) {
        GameObject newEnemy = Instantiate(enemyPrefab);
        // ...existing per-enemy positioning...
        newEnemy.GetComponent<Enemy>().speed = 1.5f + (GameManager.wave - 1) * 0.5f;
    }
    GameManager.enemiesRemaining = count;
}
```

Level 3 - no single combined code example:

This week's Level 3 is a "choose ONE" menu of five independent stretch features (destructible shields, a power-up, a bonus enemy, formation-marching movement, or a title/game-over screen) rather than the usual set of small, combinable stretch goals - so a single "all goals combined" code block doesn't fit Rule 14's usual model, and isn't provided here. Full worked solutions for all five options, across the active engines, are planned for the Lesson 15 wrap-up "Complete Codebase" send-home materials instead. Use the Talking Points above and the Handout's Level 3 menu to guide students toward whichever option they choose.

Common Student Mistakes

- Writing the formula twice (once for the loop, once for enemiesRemaining) instead of storing it in a local variable - works if both are correct, but an easy place for a typo to cause disagreement.
- Forgetting GetComponent<Enemy>() when trying to set the new enemy's speed field, attempting to set it on the wrong object type.
- Leaving Enemy.cs's default public float speed = 1.5f as the ONLY place speed is set, forgetting the new per-instance override in SpawnEnemies() entirely.